Started on	Tuesday, 10 January 2023, 8:04 PM
State	Finished
Completed on	Tuesday, 10 January 2023, 8:06 PM
Time taken	2 mins 30 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 2.00 out of 2.00

What is the difference between UNIX sockets and IP sockets?

Select one:

a. there is no difference; they refer to the same thing

b. IP sockets are mainly used for communication between processes on the same host, while UNIX sockets allow communication between processes over the network

c. IP sockets use a file on the server's disk as the address

d. UNIX sockets allow communication between processes on the same host, while IP sockets are mainly used for communication between processes over the network ✔

Unix sockets are used in communication between processes on the same host. They use a "file path" as the address (e.g /path/to/my_unix_socket). Please note this is not a regular file, it is a socket-type file which means it doesn't contain data on the disk. IP sockets are a mechanism allowing communication between processes over the network. They also allow communication between processes on the same host, but since the latency is increased when used in this manner, it is advisable not to use them like this.

The correct answer is: UNIX sockets allow communication between processes on the same host, while IP sockets are mainly used for communication between processes over the network

Question **2**

Correct

Mark 2.00 out of 2.00

What is the following sequence equivalent to in a single-threaded process?

```
close(1);
int fd = open("file", O_WRONLY);
```

Select one:

a.
```
int fd = open("file", O_WRONLY);
dup2(1, fd);
```

b.
```
int fd = open("file", O_WRONLY);
dup(1);
```

c.
```
int fd = open("file", O_WRONLY);
dup2(fd, 1);
```
✔

d.
```
int fd = open("file", O_WRONLY);
dup(fd);
```

`dup2` has the following signature:

```
int dup2(int oldfd, int newfd);
```

When using `dup2`, if `newfd` is open, it will be closed before being reused. Therefore, in our example, 1 will be closed, then it will be allocated again to refer to the same open file description as the descriptor `oldfd`, in our case, fd. In this way, writing to stdout (fd 1) will be equivalent to writing to fd 3 (the file "file").

The correct answer is: `int fd = open("file", O_WRONLY);`
`dup2(fd, 1);`

Question **3**

Correct

Mark 2.00 out of 2.00

Which file descriptor is associated by default to `stdin`?

Select one:

a. It differs from process to process

b. 0 ✔

c. 1

d. 2

stdin has file descriptor 0 associated by default. stdout has fd 1, and stderr has fd 2.

The correct answer is: 0

Question **4**

Correct

Mark 2.00 out of 2.00

Why would we prefer to use a named pipe instead of an anonymous pipe?

Select one:

- a. named pipes allow communication between two processes that are not related, while anonymous pipes don't ✔
- b. named pipes have better performance because they don't write data to a file
- c. named pipes are much faster because they are nonblocking
- d. named pipes read and write from/to the same end of the pipe, so fewer file descriptors are needed

Named pipes help overcome the biggest drawback of anonymous pipes, which only allow IPC between related processes. They do that by using some filesystem support, which sacrifices little to no performance.

The correct answer is: named pipes allow communication between two processes that are not related, while anonymous pipes don't

Question **5**

Correct

Mark 2.00 out of 2.00

Suppose we have a process that wants to open a file. We know only the standard file descriptors (0, 1, and 2) are open in that process. What file descriptor do we expect the `open` call to return?

Select one:

- a. it will return -1, it cannot allocate any more descriptors
- b. a random file descriptor
- c. 3 ✔
- d. 1023

The lowest available file descriptor will be assigned, which is, in this case, 3.

The correct answer is: 3

◄ Curs 10: I/O (partea a 3-a): Recap

Jump to...

Curs 12: Interacțiunea între aplicații (partea a 2-a) ►

Started on	Tuesday, 10 January 2023, 8:01 PM
State	Finished
Completed on	Tuesday, 10 January 2023, 8:03 PM
Time taken	2 mins 8 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 2.00 out of 2.00

How many File Descriptor Tables are there in an Operating System?

Select one:

- a. 0
- b. 2 per each thread in a process
- c. 1
- d. 1 per process ✔

Each process has its own File Descriptor Table. The threads of a process share the File Descriptor Table of the process, so there is one FDT per process.

The correct answer is: 1 per process

Question **2**

Correct

Mark 2.00 out of 2.00

What is an advantage of an asynchronous operation?

Select one:

- a. it is nonblocking, and we can send more data with it
- b. it is nonblocking, and it immediately returns the result of the operation
- c. it is blocking, and it immediately returns the result of the operation
- d. it is nonblocking, and you can do other operations while it executes ✔

When performing an asynchronous operation, the given function returns immediately, and the program continues its execution. When the operation succeeds, a notification may be sent out to the program, or the program can periodically check the state of the operation.

The correct answer is: it is nonblocking, and you can do other operations while it executes

Question **3**

Correct

Mark 2.00 out of 2.00

Which of the following is an advantage of using `dup2` instead of `dup`?

```
int dup(int oldfd);
int dup2(int oldfd, int newfd);
```

Select one:

- a. `dup2` is executed in user space
- b. `dup2` allows duplicating a file descriptor into a file descriptor which doesn't have to be sequentially ascending to existing descriptors ✔
- c. `dup2` performs slow operations in user space and fast operations in kernel space
- d. `dup2` is faster

The `dup2` call makes the file descriptor called newfd refer to the same open file description as the descriptor oldfd. It allows duplicating on a position that is not sequentially ascending to existing descriptors.

The correct answer is: `dup2` allows duplicating a file descriptor into a file descriptor which doesn't have to be sequentially ascending to existing descriptors

Question **4**

Correct

Mark 2.00 out of 2.00

When is an Open File Structure deleted?

Select one:

- a. When the time from the time to live variable expires
- b. Any time we call the `close` function
- c. Never
- d. When no more handles reference it ✔

When we call the `close` function, the reference count in the Open File Structure is decremented. The Open File Structure is deleted if the reference count is 0, which happens when no handles reference it anymore.

The correct answer is: When no more handles reference it

Question **5**

Correct

Mark 2.00 out of 2.00

When is `read` blocking?

Select one:

- a. when the buffer is full
- b. any time we try to read something
- c. never
- d. when the buffer is empty ✔

`read` blocks when its associated buffer is empty because it doesn't have anything available to read.

The correct answer is: when the buffer is empty

◄ Curs 10: I/O (partea a 3-a): Recap

Jump to...

Curs 12: Interacțiunea între aplicații (partea a 2-a) ►

Started on	Tuesday, 10 January 2023, 7:50 PM
State	Finished
Completed on	Tuesday, 10 January 2023, 7:54 PM
Time taken	3 mins 37 secs
Grade	**8.00** out of 10.00 (**80**%)

Question **1**

Correct

Mark 2.00 out of 2.00

How many File Descriptor Tables are there in an Operating System?

Select one:

- a. 0
- b. 1
- c. 2 per each thread in a process
- d. 1 per process ✔

Each process has its own File Descriptor Table. The threads of a process share the File Descriptor Table of the process, so there is one FDT per process.

The correct answer is: 1 per process

Question **2**

Correct

Mark 2.00 out of 2.00

What is the difference between UNIX sockets and IP sockets?

Select one:

- a. there is no difference; they refer to the same thing
- b. IP sockets use a file on the server's disk as the address
- c. UNIX sockets allow communication between processes on the same host, while IP sockets are mainly used for communication between processes over the network ✔
- d. IP sockets are mainly used for communication between processes on the same host, while UNIX sockets allow communication between processes over the network

Unix sockets are used in communication between processes on the same host. They use a "file path" as the address (e.g /path/to/my_unix_socket). Please note this is not a regular file, it is a socket-type file which means it doesn't contain data on the disk.
IP sockets are a mechanism allowing communication between processes over the network. They also allow communication between processes on the same host, but since the latency is increased when used in this manner, it is advisable not to use them like this.

The correct answer is: UNIX sockets allow communication between processes on the same host, while IP sockets are mainly used for communication between processes over the network

Question **3**

Correct

Mark 2.00 out of 2.00

When is `write` blocking?

Select one:

- a. when the buffer is empty
- b. any time we try to write something
- c. when the buffer is full ✔
- d. never

`write` blocks when its associated buffer is full because it no longer has any space to write.

The correct answer is: when the buffer is full

Question **4**

Incorrect

Mark 0.00 out of 2.00

What is the advantage of libc buffering?

Select one:

- a. it makes printf faster because it performs one `write()` syscall per call to `printf()`
- b. it allows us to write more bytes using printf
- c. it doesn't require synchronizing the disk after every `write()` ✖
- d. it reduces the number of context switches between user space and kernel space

libc buffering reduces the number of context switches between user space and kernel space. For this, it copies the string passed to printf() to an internal buffer inside libc. This buffer will be written at a given time (like when the buffer is full, for example).

The correct answer is: it reduces the number of context switches between user space and kernel space

Question **5**

Correct

Mark 2.00 out of 2.00

Which of the following is an example of a device file with a corresponding physical device?

Select one:

- a. /dev/random
- b. /dev/null
- c. /dev/zero
- d. /dev/sda1 ✔

Out of the choices above, `/dev/random`, `/dev/zero`, and `/dev/null` are all pseudo-devices, they don't have a corresponding physical device, and we can fully cover their behavior using software.
`/dev/sda1`, on the other hand, refers to the SCSI and SATA controllers devices.

The correct answer is: /dev/sda1

Started on	Tuesday, 10 January 2023, 9:17 PM
State	Finished
Completed on	Tuesday, 10 January 2023, 9:22 PM
Time taken	4 mins 53 secs
Grade	**8.00** out of 10.00 (**80**%)

Question **1**

Correct

Mark 2.00 out of 2.00

Which of the following is an advantage of using `dup2` instead of `dup`?

```
int dup(int oldfd);
int dup2(int oldfd, int newfd);
```

Select one:

- a. `dup2` is executed in user space
- b. `dup2` allows duplicating a file descriptor into a file descriptor which doesn't have to be sequentially ascending to existing descriptors ✔
- c. `dup2` is faster
- d. `dup2` performs slow operations in user space and fast operations in kernel space

The `dup2` call makes the file descriptor called newfd refer to the same open file description as the descriptor oldfd. It allows duplicating on a position that is not sequentially ascending to existing descriptors.

The correct answer is: `dup2` allows duplicating a file descriptor into a file descriptor which doesn't have to be sequentially ascending to existing descriptors

Question **2**

Correct

Mark 2.00 out of 2.00

Given a file called "file", which has 5 bytes in size, what happens after the following sequence?

```
int fd = open("file", O_WRONLY);
ftruncate(fd, 10);
```

where `ftruncate` has the following signature:

```
int ftruncate(int fd, off_t length);
```

Select one:

- a. "file" now has 10 bytes in size, and its content is duplicated so it can reach this size
- b. nothing happens, "file" still has 5 bytes in size because we cannot truncate it with a larger size
- c. "file" will be deleted because it was truncated with a larger size than it initially had
- d. "file" now has 10 bytes in size, and it is "filled" with binary zeros to the indicated `length` ✔

When using `ftruncate`, if the file size is smaller than `length`, the file is extended and "filled" with binary zeros to the indicated `length`.

The correct answer is: "file" now has 10 bytes in size, and it is "filled" with binary zeros to the indicated `length`

Question **3**

Correct

Mark 2.00 out of 2.00

When is an Open File Structure deleted?

Select one:

- a. Never
- b. Any time we call the `close` function
- c. When the time from the time to live variable expires
- d. When no more handles reference it ✔

When we call the `close` function, the reference count in the Open File Structure is decremented. The Open File Structure is deleted if the reference count is 0, which happens when no handles reference it anymore.

The correct answer is: When no more handles reference it

Question **4**

Incorrect

Mark 0.00 out of 2.00

When is `read` blocking?

Select one:

- a. any time we try to read something
- b. when the buffer is empty
- c. never
- d. when the buffer is full ✖

`read` blocks when its associated buffer is empty because it doesn't have anything available to read.

The correct answer is: when the buffer is empty

Question **5**

Correct

Mark 2.00 out of 2.00

What is an advantage of zero-copy?

Select one:

- a. data is copied into the app's internal buffer, so we have caching
- b. it performs a single context switch ✔
- c. it doesn't perform any context switching
- d. data is copied multiple times, so we have redundancy

The correct answer is: it performs a single context switch

◄ Curs 10: I/O (partea a 3-a): Recap

Jump to...

Curs 12: Interacțiunea între aplicații (partea a 2-a) ►

Started on	Tuesday, 10 January 2023, 7:56 PM
State	Finished
Completed on	Tuesday, 10 January 2023, 8:00 PM
Time taken	4 mins 25 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 2.00 out of 2.00

Given a file called "file", which has 5 bytes in size, what happens after the following sequence?

```
int fd = open("file", O_WRONLY);
ftruncate(fd, 10);
```

where `ftruncate` has the following signature:

```
int ftruncate(int fd, off_t length);
```

Select one:

- a. "file" now has 10 bytes in size, and its content is duplicated so it can reach this size
- b. "file" will be deleted because it was truncated with a larger size than it initially had
- c. "file" now has 10 bytes in size, and it is "filled" with binary zeros to the indicated `length` ✔
- d. nothing happens, "file" still has 5 bytes in size because we cannot truncate it with a larger size

When using `ftruncate`, if the file size is smaller than `length`, the file is extended and "filled" with binary zeros to the indicated `length`.

The correct answer is: "file" now has 10 bytes in size, and it is "filled" with binary zeros to the indicated `length`

Question **2**

Correct

Mark 2.00 out of 2.00

Why would we prefer to use a named pipe instead of an anonymous pipe?

Select one:

- a. named pipes are much faster because they are nonblocking
- b. named pipes have better performance because they don't write data to a file
- c. named pipes read and write from/to the same end of the pipe, so fewer file descriptors are needed
- d. named pipes allow communication between two processes that are not related, while anonymous pipes don't ✔

Named pipes help overcome the biggest drawback of anonymous pipes, which only allow IPC between related processes. They do that by using some filesystem support, which sacrifices little to no performance.

The correct answer is: named pipes allow communication between two processes that are not related, while anonymous pipes don't

Question **3**

Correct

Mark 2.00 out of 2.00

What is an advantage of an asynchronous operation?

Select one:

- a. it is nonblocking, and you can do other operations while it executes ✔
- b. it is blocking, and it immediately returns the result of the operation
- c. it is nonblocking, and it immediately returns the result of the operation
- d. it is nonblocking, and we can send more data with it

When performing an asynchronous operation, the given function returns immediately, and the program continues its execution. When the operation succeeds, a notification may be sent out to the program, or the program can periodically check the state of the operation.

The correct answer is: it is nonblocking, and you can do other operations while it executes

Question **4**

Correct

Mark 2.00 out of 2.00

Which of the following is an advantage of using `dup2` instead of `dup`?

```
int dup(int oldfd);
int dup2(int oldfd, int newfd);
```

Select one:

- a. `dup2` allows duplicating a file descriptor into a file descriptor which doesn't have to be sequentially ascending to existing descriptors ✔
- b. `dup2` performs slow operations in user space and fast operations in kernel space
- c. `dup2` is executed in user space
- d. `dup2` is faster

The `dup2` call makes the file descriptor called newfd refer to the same open file description as the descriptor oldfd. It allows duplicating on a position that is not sequentially ascending to existing descriptors.

The correct answer is: `dup2` allows duplicating a file descriptor into a file descriptor which doesn't have to be sequentially ascending to existing descriptors

Question **5**

Correct

Mark 2.00 out of 2.00

What does the following sequence do?

```
int fd = open("file", O_WRONLY);
dup2(fd, 1);
```

Select one:

- a. Redirects the process' `stdin` to the file "file"
- b. Redirects the process' `stdout` to file "file" ✔
- c. Redirects writing to "file" to write to `stdout`
- d. Redirects the process' `stderr` to file "file"

The file descriptor with the value 1 (`stdout`) is replaced by the file descriptor `fd`, which points to the file "file", therefore every time we try to write to `stdout`, we will write to the file "file".

The correct answer is: Redirects the process' `stdout` to file "file"

◄ Curs 10: I/O (partea a 3-a): Recap

Jump to...

Curs 12: Interacțiunea între aplicații (partea a 2-a) ►